

Package ‘rbiogeme’

September 29, 2026

Type Package

Title Interface to 'Biogeme'

Version 0.1.2

Author Michel Bierlaire [aut, cre]

Maintainer Michel Bierlaire <michel.bierlaire@epfl.ch>

Description Uses the 'Python' implementation of 'Biogeme' as the numerical backend for specifying and estimating discrete-choice models in R. The default native requirement is 'biogeme==3.3.5'.

Depends R (>= 4.3.0)

Imports reticulate (>= 1.41.0), stats, utils

Suggests testthat (>= 3.0.0), knitr, rmarkdown

SystemRequirements Python (>= 3.12), Biogeme (== 3.3.5)

License MIT + file LICENSE

Encoding UTF-8

Config/testthat/edition 3

VignetteBuilder knitr

URL <https://github.com/michelbierlaire/rbiogeme>

BugReports <https://github.com/michelbierlaire/rbiogeme/issues>

Date 2026-09-02

Config/roxygen2/version 8.1.0

NeedsCompilation no

Repository CRAN

Date/Publication 2026-09-29 14:20:07 UTC

Contents

rbiogeme-package	4
abs.biogeme_expression	5
assisted_specification	6
bayesian_estimate	7
bayesian_posterior_mean_by_observation	8
bayesian_stored_variables	8
biogeme_beta	9
biogeme_catalog	10
biogeme_catalog_controller	10
biogeme_check	11
biogeme_confidence_intervals	11
biogeme_config	12
biogeme_control	13
biogeme_database	15
biogeme_database_columns	15
biogeme_database_define_variable	16
biogeme_database_extract_rows	17
biogeme_database_filtered_row_count	17
biogeme_database_has_column	18
biogeme_database_is_panel	18
biogeme_database_materialize	19
biogeme_database_nrow	19
biogeme_database_panel	20
biogeme_database_remove	20
biogeme_database_row_ids	21
biogeme_database_segmentation	22
biogeme_diagnostics	22
biogeme_draws	23
biogeme_error_details	23
biogeme_general_statistics	24
biogeme_generic_alt_specific_catalogs	24
biogeme_max	25
biogeme_mdcev_model	26
biogeme_min	27
biogeme_model	28
biogeme_model_parameters	29
biogeme_native_parameter_names	30
biogeme_panel_database	30
biogeme_prior	31
biogeme_python	31
biogeme_sampling_partition	32
biogeme_segmentation	32
biogeme_segmentation_catalogs	33
biogeme_setup	34
boxcox	35
catalog_configuration_ids	35

check_derivatives	36
check_monte_carlo_stability	36
count_number_of_specifications	37
cross_nested_logit_correlation	38
cross_nested_logit_model	39
cross_nested_log_probability	40
cross_nested_nest	40
cross_nested_nests	41
cross_nested_probability	41
cross_nested_sparsity_report	42
cross_variable	43
derive	43
distributed_parameter	44
draw	44
Elem	45
estimate	45
estimate_catalog	46
estimate_configuration	47
estimate_or_load	48
estimate_sampled_alternatives	49
evaluate_biogeme_expression	50
evaluate_biogeme_expression_c	51
integrate_normal	52
linear_term	52
linear_utility	53
logit_log_probability	53
logit_model	54
logit_probability	55
logzero	56
mdcev_estimate	56
mdcev_forecast	57
mdcev_forecast_describe	58
mdcev_generate_epsilon	59
mdcev_parameter_table	59
mdcev_short_summary	60
mdcev_validate_forecast	60
monte_carlo	61
nested_endogenous_sampling_log_probability	62
nested_logit_correlation	62
nested_logit_model	63
nested_log_probability	64
nested_nest	65
nested_nests	65
nested_probability	66
normal_cdf	66
normal_pdf	67
ordered_logit_log_probability	67
ordered_probit_log_probability	68

panel_likelihood_trajectory	68
pareto_post_processing	69
piecewise	70
predict.biogeme_fit	71
profile_jax	72
quick_estimate	73
random_variable	74
read_results	74
safe_exp	75
sampld_alternatives_model	75
sampling_segment_sizes	76
save_results	77
segment_beta	77
simulate	78
simulate_bayesian	79
simulate_single_formula	79
sqrt.biogeme_expression	80
swissmetro_b01b_model	81
swissmetro_data	81
swissmetro_mnl_model	82
validate	83
validate_model	83
variable	84
Index	86

rbiogeme-package	<i>Interface to Biogeme</i>
------------------	-----------------------------

Description

rbiogeme is a complete R-facing interface to the native Biogeme estimation engine. Data, parameters, expressions, models, estimation controls, and post-estimation operations are specified from R. The bridge compiles the complete expression tree once before native numerical work begins.

Details

Start a new session with `biogeme_setup()` to provision or verify the runtime and receive corrective actions. Configure the runtime with `biogeme_config()` before the first call that initializes Python when selecting an existing interpreter. The default runtime is lazy. A typical workflow is to create a numeric data frame, wrap it with `biogeme_database()`, build symbolic expressions with `variable()` and `biogeme_beta()`, construct a specialized or generic model, and call `estimate()`. The returned fit supports the ordinary R methods `summary()`, `coef()`, `vcov()`, `logLik()`, and `nobs()`.

Specialized constructors cover multinomial, nested, cross-nested, panel, Bayesian, MDCEV, hybrid-choice, catalog, assisted-specification, and sampled-alternative workflows. Generic models use a complete likelihood expression and can additionally hold a probability, named simulation expressions, weights, panel aggregation, draw metadata, subsets, and parameter overrides.

Expressions are symbolic. Arithmetic, comparisons, logical operators, transformations, probability functions, draws, integration nodes, and derivatives create neutral expression nodes; they do not evaluate a local R likelihood. Native Biogeme performs compilation, differentiation, integration, optimization, simulation, and reporting.

The expression operators `+`, `-`, `*`, `/`, and `^` build arithmetic nodes. Comparisons `==`, `!=`, `<`, `<=`, `>`, and `>=` build indicator nodes. Logical conjunction, disjunction, and negation use `\&`, `|`, and `!`. Use `logzero()` and `safe_exp()` when the native numerically safe form is required.

Use `estimate()` for fresh estimation. Use `estimate_or_load()` only when explicit YAML loading or recycling is desired, and choose a fresh temporary output directory for equivalence tests. Random-draw and sampling examples should fix the native draw design and seed and document any remaining simulation noise.

Start with `?biogeme_setup` or `?biogeme_check`, then read `vignette("getting-started", package = "rbiogeme")` and `vignette("modeling-workflows", package = "rbiogeme")` for the R syntax and the complete workflow. Advanced Bayesian, Monte Carlo, MDCEV, catalog, hybrid-choice, and sampled-alternative examples are in `vignette("advanced-models", package = "rbiogeme")`.

The whole package was implemented by ChatGPT 5.6 Luna under the supervision of Michel Bierlaire.

Value

Depending on the function, a database, expression, model, diagnostics list, estimation result, or result-derived object.

See Also

`biogeme_config`, `biogeme_database`, `biogeme_model`, `logit_model`, `estimate`, `simulate`, `summary.biogeme_fit`, `coef.biogeme_fit`, `vcov.biogeme_fit`, `logLik.biogeme_fit`

`abs.biogeme_expression`

Absolute value of a Biogeme expression

Description

Absolute value of a Biogeme expression

Usage

```
## S3 method for class 'biogeme_expression'
abs(x)
```

Arguments

`x` A scalar or Biogeme expression.

Value

A Biogeme expression.

assisted_specification

Run native Biogeme assisted specification

Description

The complete catalog search, quick-estimation objective evaluation, Pareto persistence, and final re-estimation are delegated to native Biogeme. The R interface accepts named objective and validity descriptors rather than exposing Python callback objects. The selected validity rule is created and executed inside the Python bridge between native estimates.

Usage

```
assisted_specification(
  model,
  objectives = "loglikelihood_dimension",
  pareto_file_name = NULL,
  model_name = "rbiogeme_assisted",
  controls = list(),
  force = TRUE,
  control = NULL,
  validity = NULL
)

run_assisted_specification(
  model,
  objectives = "loglikelihood_dimension",
  pareto_file_name = NULL,
  model_name = "rbiogeme_assisted",
  controls = list(),
  force = TRUE,
  control = NULL,
  validity = NULL
)
```

Arguments

model	A biogeme_model containing catalog expressions.
objectives	Native objective preset. Supported values are "loglikelihood_dimension" and "aic_bic_dimension".
pareto_file_name	Explicit path of the native Pareto checkpoint file. It has no default because native checkpoints are persistent files.
model_name	Native Biogeme model name prefix.
controls	Named list of native Biogeme controls.
force	Whether to remove the named Pareto checkpoint and start fresh.

control	Optional biogeme_control() object; an alias for controls.
validity	Optional native validity-rule name. Currently supported is "negative_time_cost", matching the predicate in the native assisted Swissmetro example.

Value

An object of class biogeme_assisted_fit containing native final results, the summary table, descriptions, and Pareto metadata.

bayesian_estimate	<i>Estimate a model with native Bayesian inference</i>
-------------------	--

Description

The complete R expression tree is compiled once, then native Biogeme builds and samples the PyMC model. The returned R object contains the native posterior summary and the paths of any generated NetCDF/YAML/HTML files; posterior draws are not represented as manually managed Python objects.

Usage

```
bayesian_estimate(
  model,
  model_name = "rbiogeme_model",
  controls = list(),
  starting_values = NULL,
  control = NULL
)
```

Arguments

model	A biogeme_model.
model_name	Native Biogeme model name.
controls	Named native Biogeme controls.
starting_values	Optional named numeric vector of starting values.
control	Optional biogeme_control() object; an alias for controls.

Details

NetCDF and YAML output are enabled by default for this operation. Additional Bayesian controls such as bayesian_draws, warmup, chains, target_accept, calculate_likelihood, calculate_waic, calculate_loo, and mcmc_sampling_strategy are passed to native Biogeme through controls. Because those default Bayesian files are persistent, supply an explicit output_directory through [biogeme_control\(\)](#) (or disable both output types explicitly).

Value

An object of class `biogeme_bayesian_fit` containing the native posterior summary and output paths.

`bayesian_posterior_mean_by_observation`

Retrieve native posterior means by observation

Description

Loads the NetCDF result with Biogeme's public `BayesianResults` API and delegates the observation-level posterior mean calculation to native Biogeme. No posterior-draw or ArviZ object is exposed to R.

Usage

```
bayesian_posterior_mean_by_observation(bayesian_results, variable_name,
  control = NULL)
```

Arguments

<code>bayesian_results</code>	A <code>biogeme_bayesian_fit</code> object or one native Bayesian NetCDF result path.
<code>variable_name</code>	Name of a stored posterior variable with one observation dimension in addition to chain and draw.
<code>control</code>	Optional native Biogeme control list. Likelihood, WAIC, and LOO calculations are disabled by default.

Value

A data frame indexed by the native observation coordinate, with one column named after `variable_name`.

`bayesian_stored_variables`

Report variables stored in native Bayesian results

Description

Returns the serialized result of native Biogeme's `BayesianResultsSummary.report_stored_variables()` operation. No PyMC or ArviZ object is exposed to R.

Usage

```
bayesian_stored_variables(x)
```

Arguments

x A biogeme_bayesian_fit object returned by bayesian_estimate().

Value

A data frame describing each native stored variable by result group, variable name, dimensions, and shape.

biogeme_beta	<i>Create a Biogeme parameter</i>
--------------	-----------------------------------

Description

Create a Biogeme parameter

Usage

```
biogeme_beta(
  name,
  start = 0,
  lower = NULL,
  upper = NULL,
  fixed = FALSE,
  sigma_prior = 5,
  prior = NULL
)
```

Arguments

name	Parameter name.
start	Initial value.
lower	Optional lower bound.
upper	Optional upper bound.
fixed	Logical value indicating whether the parameter is fixed.
sigma_prior	Standard deviation of the native default normal prior.
prior	Optional declarative prior from biogeme_prior() . It is compiled into a native PyMC prior factory; R callbacks are not accepted.

Details

Parameter names are preserved exactly by the bridge and are therefore part of the equivalence contract with native Biogeme. Bounds and fixed are passed to the native parameter definition.

Value

A parameter expression.

Examples

```
b_cost <- biogeme_beta("b_cost", start = -1, upper = 0)
b_cost
```

biogeme_catalog	<i>Create a native-compatible expression catalog</i>
-----------------	--

Description

Create a native-compatible expression catalog

Usage

```
biogeme_catalog(name, expressions, controller = NULL)

catalog(name, expressions, controller = NULL)
```

Arguments

name	Catalog name.
expressions	Non-empty named list of alternative expressions.
controller	Optional biogeme_catalog_controller() shared by catalogs.

Value

A catalog expression compiled to native Biogeme.

biogeme_catalog_controller	<i>Create a neutral controller for one or more expression catalogs</i>
----------------------------	--

Description

Catalog controllers select the same named specification in every catalog that shares the controller. The object remains an R specification until the complete expression is compiled to native Biogeme.

Usage

```
biogeme_catalog_controller(name, specification_names, selected_name = NULL)

catalog_controller(name, specification_names, selected_name = NULL)
```

Arguments

name Controller name.
 specification_names Ordered, unique catalog specification names.
 selected_name Optional specification selected before native compilation.

Value

A neutral catalog controller.

biogeme_check	<i>Check whether rbiogeme is ready to run a model</i>
---------------	---

Description

This is the runtime-only validation used by `biogeme_setup()` and is also useful when the environment has already been configured. It initializes the configured runtime, verifies the minimum R and Python versions, checks that native Biogeme can be imported, and compares its version with the configured requirement when that requirement pins an exact version. The check does not construct or estimate a model.

Usage

```
biogeme_check(verbose = interactive())
```

Arguments

verbose If TRUE, print the check and its actionable messages.

Value

An object of class `biogeme_check` with elements `ready`, `diagnostics`, and `issues`. `issues` is a data frame with columns `check`, `status`, `message`, and `action`.

biogeme_confidence_intervals	<i>Calculate native simulation confidence intervals</i>
------------------------------	---

Description

Native Biogeme performs the repeated simulation and quantile calculation. R supplies ordinary parameter mappings and receives two ordinary data frames; no native simulation or result objects are exposed.

Usage

```
biogeme_confidence_intervals(
  model,
  beta_values,
  expressions = NULL,
  interval_size = 0.9,
  control = NULL
)
```

Arguments

model	A biogeme_model.
beta_values	A non-empty list of named finite numeric parameter vectors, typically obtained from a fitted result's bootstrap field.
expressions	Optional named list of expressions. When omitted, the model's simulations list is used.
interval_size	Confidence interval size, strictly between zero and one. The native default is 0.9.
control	Optional <code>biogeme_control()</code> object.

Value

A list with left and right data frames and native metadata.

biogeme_config	<i>Configure the Python runtime used by rbiogeme</i>
----------------	--

Description

Configuration must be performed before Python is initialized in the R session. The default requirement is `biogeme==3.3.5`; use this function to select a different compatible Biogeme requirement explicitly.

Usage

```
biogeme_config(python = NULL, biogeme_requirement = NULL, debug = NULL)
```

Arguments

python	Optional Python executable. If NULL, reticulate selects the interpreter according to its normal configuration rules.
biogeme_requirement	Python requirement passed to <code>reticulate::py_require()</code> .
debug	If TRUE, preserve the Python traceback in Biogeme error conditions.

Details

`biogeme_config(python = ...)` selects an existing Python interpreter; it does not install Biogeme into that interpreter. If `python` is omitted, `reticulate` can provision the configured requirement in its managed environment.

Value

The current configuration, invisibly when changes are requested and visibly otherwise.

<code>biogeme_control</code>	<i>Define estimation and simulation controls</i>
------------------------------	--

Description

Unspecified fields are omitted so native Biogeme defaults remain in force. Additional named arguments are passed through to the native bridge.

Usage

```
biogeme_control(
  model_name = NULL,
  output_directory = NULL,
  seed = NULL,
  numerically_safe = NULL,
  second_derivatives = NULL,
  second_derivatives_percentage = NULL,
  optimization_algorithm = NULL,
  number_of_draws = NULL,
  draw_type = NULL,
  draw_seed = NULL,
  bootstrap_samples = NULL,
  user_notes = NULL,
  variance_covariance_type = NULL,
  save_iterations = NULL,
  generate_html = NULL,
  generate_yaml = NULL,
  validation_folds = NULL,
  ...
)
```

Arguments

<code>model_name</code>	Optional native Biogeme model name.
<code>output_directory</code>	Optional explicit directory for native files. It is required whenever a control requests native HTML, YAML, NetCDF, pickle, or iteration output; no operation writes to the working directory by default.

seed	Optional random seed.
numerically_safe	Optional numerical-safety flag.
second_derivatives	Optional second-derivative policy.
second_derivatives_percentage	Optional native percentage of iterations using analytical second derivatives.
optimization_algorithm	Optional native algorithm name.
number_of_draws	Optional number of simulation draws.
draw_type	Optional default draw type metadata.
draw_seed	Optional draw seed (mapped to native seed).
bootstrap_samples	Optional bootstrap sample count.
user_notes	Optional notes included in native reports/results.
variance_covariance_type	Optional result covariance selection, such as "BHHH" or "Bootstrap".
save_iterations	Optional iteration-file persistence flag.
generate_html	Optional HTML-report flag.
generate_yaml	Optional YAML-report flag.
validation_folds	Optional validation-fold count.
...	Additional native Biogeme controls.

Details

Unspecified values are omitted. This lets native Biogeme apply its normal defaults. The control object can be stored in a model or passed to an estimation or simulation operation; it is not a Python object.

Value

A named list of controls.

Examples

```
biogeme_control(
  seed = 1234,
  generate_html = FALSE,
  generate_yaml = FALSE,
  save_iterations = FALSE
)
```

biogeme_database	<i>Create a Biogeme database</i>
------------------	----------------------------------

Description

The database remains an R object until a model is compiled. Data are copied and validated at construction time, so later modifications to the original data frame do not alter the model database.

Usage

```
biogeme_database(name, data)
```

Arguments

name	Database name.
data	Numeric R data frame.

Details

The input must contain only numeric, named, non-empty columns. The original row identifiers are retained separately from the data columns and are carried through native filtering and row-extraction operations.

Value

An object of class `biogeme_database`.

Examples

```
database <- biogeme_database(
  "demo",
  data.frame(choice = c(1, 2), income = c(10, 20))
)
biogeme_database_columns(database)
```

biogeme_database_columns	<i>Return the columns available in a Biogeme database</i>
--------------------------	---

Description

Return the columns available in a Biogeme database

Usage

```
biogeme_database_columns(database)
```

Arguments

database A biogeme_database object.

Value

A character vector of column names.

biogeme_database_define_variable

Define a derived database variable using a complete Biogeme expression

Description

Define a derived database variable using a complete Biogeme expression

Usage

```
biogeme_database_define_variable(database, name, expression)
```

```
database_define_variable(database, name, expression)
```

```
define_variable(database, name, expression)
```

Arguments

database A biogeme_database object.
 name Name of the derived column.
 expression Complete Biogeme expression.

Details

The expression is retained symbolically and is compiled by the Python bridge together with the model. It is not evaluated by an R callback during estimation or simulation.

Value

A new database specification.

Examples

```
database <- biogeme_database(
  "demo",
  data.frame(time = c(10, 12), cost = c(5, 6))
)
database <- biogeme_database_define_variable(
  database, "time_cost", variable("time") / variable("cost")
)
biogeme_database_columns(database)
```

`biogeme_database_extract_rows`*Extract observations from a Biogeme database*

Description

This is the R equivalent of native `Database.extract_rows()`. Indices are deliberately one-based, as in the rest of the R interface, and the original row identifiers are retained in the returned database.

Usage

```
biogeme_database_extract_rows(database, indices)
```

```
database_extract_rows(database, indices)
```

Arguments

`database` A `biogeme_database` object.

`indices` Unique positive one-based row indices, in the order to keep.

Value

A new `biogeme_database` containing the selected observations.

`biogeme_database_filtered_row_count`*Return the number of observations excluded by native database filters*

Description

Return the number of observations excluded by native database filters

Usage

```
biogeme_database_filtered_row_count(database)
```

Arguments

`database` A `biogeme_database` object.

Value

An integer, or NA until lazy filters are materialised.

`biogeme_database_has_column`*Test whether a Biogeme database contains a column*

Description

Test whether a Biogeme database contains a column

Usage

```
biogeme_database_has_column(database, column)
```

Arguments

database	A biogeme_database object.
column	Column name.

Value

A single logical value.

`biogeme_database_is_panel`*Return whether a database has a declared panel structure*

Description

Return whether a database has a declared panel structure

Usage

```
biogeme_database_is_panel(database)
```

Arguments

database	A biogeme_database object.
----------	----------------------------

Value

One logical value.

`biogeme_database_materialize`*Materialize lazy database operations through native Biogeme*

Description

Materialize lazy database operations through native Biogeme

Usage

```
biogeme_database_materialize(database)
```

Arguments

database A biogeme_database object.

Value

A database with native derived columns and filters applied.

`biogeme_database_nrow` *Return the number of rows currently represented by a database*

Description

Return the number of rows currently represented by a database

Usage

```
biogeme_database_nrow(database)
```

Arguments

database A biogeme_database object.

Value

An integer, or NA until lazy native filters are materialised.

```
biogeme_database_panel
```

Declare and validate a panel identifier

Description

Declare and validate a panel identifier

Usage

```
biogeme_database_panel(database, panel_id)
```

Arguments

database	A biogeme_database object.
panel_id	Name of the panel identifier column.

Details

The identifier must be numeric and finite. Observations belonging to one identifier must be contiguous; the constructor rejects interleaved panel rows rather than silently reordering them.

Value

A new panel database specification.

Examples

```
panel_database <- biogeme_panel_database(
  "panel_demo",
  data.frame(person = c(1, 1, 2, 2), choice = c(1, 2, 2, 1)),
  panel_id = "person"
)
biogeme_database_is_panel(panel_database)
```

```
biogeme_database_remove
```

Remove observations satisfying a Biogeme logical expression

Description

Remove observations satisfying a Biogeme logical expression

Usage

```
biogeme_database_remove(database, condition)
```

```
database_remove(database, condition)
```

```
remove_observations(database, condition)
```

Arguments

database	A biogeme_database object.
condition	Logical Biogeme expression; matching rows are removed.

Details

condition is a symbolic indicator. Rows for which it is true are removed when the database is materialized by native Biogeme.

Value

A new database specification.

Examples

```
database <- biogeme_database(
  "demo",
  data.frame(choice = c(1, 2, 1), income = c(10, 20, 30))
)
filtered <- biogeme_database_remove(database, variable("income") > 10)
biogeme_database_nrow(filtered)
```

```
biogeme_database_row_ids
```

Return the original row identifiers of a Biogeme database

Description

Return the original row identifiers of a Biogeme database

Usage

```
biogeme_database_row_ids(database)
```

Arguments

database	A biogeme_database object.
----------	----------------------------

Value

A character vector of row identifiers.

biogeme_database_segmentation

Generate a native-compatible segmentation specification from a database

Description

Generate a native-compatible segmentation specification from a database

Usage

```
biogeme_database_segmentation(database, variable, mapping, reference = NULL)
```

```
database_generate_segmentation(database, variable, mapping, reference = NULL)
```

Arguments

database	A biogeme_database object.
variable	A database variable name or expression.
mapping	Named mapping from integer values to segment names.
reference	Optional reference segment name.

Value

A biogeme_segmentation specification.

biogeme_diagnostics *Report the active R, Python, Biogeme, and numerical-library versions*

Description

This function initializes the configured runtime. For a check that catches initialization failures and returns an actionable status object, use [biogeme_check\(\)](#).

Usage

```
biogeme_diagnostics()
```

Value

A named list containing environment diagnostics.

See Also

[biogeme_check\(\)](#), [biogeme_config\(\)](#)

biogeme_draws	<i>Declare first-class draw metadata for a model</i>
---------------	--

Description

Declare first-class draw metadata for a model

Usage

```
biogeme_draws(
  name,
  draw_type = "NORMAL",
  number_of_draws = NULL,
  seed = NULL,
  matrix = NULL,
  generator = NULL
)
```

Arguments

name	Draw variable name.
draw_type	Native draw type.
number_of_draws	Optional draw count.
seed	Optional draw seed.
matrix	Optional user-supplied numeric draw matrix.
generator	Optional bridge generator name. Supported bridge-owned generators include "TRIANGULAR", "HALTON13", and "HALTON13_ANTI". The latter two match the custom base-13, skip-10 generators used by the Monte Carlo examples.

Value

A biogeme_draws object.

biogeme_error_details	<i>Return the diagnostic details attached to a Biogeme error</i>
-----------------------	--

Description

Return the diagnostic details attached to a Biogeme error

Usage

```
biogeme_error_details(error)
```

Arguments

`error` A condition inheriting from `biogeme_error`.

Value

A named list with operation, suggestion, parent, and optional Python traceback details.

`biogeme_general_statistics`

Return native Biogeme general estimation statistics

Description

Return native Biogeme general estimation statistics

Usage

`biogeme_general_statistics(fit)`

Arguments

`fit` A `biogeme_fit` object.

Value

A named list of native Biogeme statistics.

`biogeme_generic_alt_specific_catalogs`

Build synchronized generic/alternative-specific catalogs

Description

This is the neutral R counterpart of native Biogeme's `generic_alt_specific_catalogs()`. Each returned element is a named list of catalogs, one for every alternative. The catalogs share one native controller with the exact specifications `generic` and `altspec`; when segmentations are supplied, the segmentation catalogs are nested below that controller in the same order as native Biogeme.

Usage

```
biogeme_generic_alt_specific_catalogs(  
  generic_name,  
  beta_parameters,  
  alternatives,  
  potential_segmentations = NULL,  
  maximum_number = 5L  
)  
  
generic_alt_specific_catalogs(  
  generic_name,  
  beta_parameters,  
  alternatives,  
  potential_segmentations = NULL,  
  maximum_number = 5L  
)
```

Arguments

- generic_name Shared name used for the segmentation and generic/alternative-specific controllers.
- beta_parameters Non-empty list of biogeme_beta() expressions.
- alternatives At least two unique alternative names.
- potential_segmentations Optional non-empty list of segmentation objects.
- maximum_number Maximum number of potential segmentations in one option.

Value

A list of named lists of catalog expressions, one list per Beta.

biogeme_max	<i>Maximum of Biogeme expressions</i>
-------------	---------------------------------------

Description

Maximum of Biogeme expressions

Usage

```
biogeme_max(...)
```

Arguments

- ... Scalar values or Biogeme expressions.

Value

A Biogeme expression.

biogeme_mdcev_model	<i>Construct an MDCEV model specification</i>
---------------------	---

Description

This is a declarative model object. The alternative utilities, shape parameters, and observed consumptions remain neutral R expression trees until the Python bridge sends the complete specification to the native Biogeme engine. The likelihood and forecasting algorithms are therefore provided by Biogeme itself.

Usage

```
biogeme_mdcev_model(
  database,
  model_type = c("gamma_profile", "generalized", "translated", "non_monotonic"),
  baseline_utilities,
  gamma_parameters,
  alpha_parameters = NULL,
  mu_utilities = NULL,
  scale_parameter = NULL,
  prices = NULL,
  weights = NULL,
  number_of_chosen_alternatives,
  consumed_quantities,
  subset = NULL,
  control = NULL
)

mdcev_model(
  database,
  model_type = c("gamma_profile", "generalized", "translated", "non_monotonic"),
  baseline_utilities,
  gamma_parameters,
  alpha_parameters = NULL,
  mu_utilities = NULL,
  scale_parameter = NULL,
  prices = NULL,
  weights = NULL,
  number_of_chosen_alternatives,
  consumed_quantities,
  subset = NULL,
  control = NULL
)
```

Arguments

database	A <code>biogeme_database()</code> object.
model_type	One of "gamma_profile", "generalized", "translated", or "non_monotonic".
baseline_utilities	Non-empty named list of baseline utility expressions, keyed by integer alternative code.
gamma_parameters	Named list of gamma expressions, keyed by the same alternatives. One value may be NULL to declare an outside good.
alpha_parameters	Named list of alpha expressions. Required for the generalized, translated, and non-monotonic variants.
mu_utilities	Named list of non-monotonic utility expressions.
scale_parameter	Optional scale expression.
prices	Optional named list of price expressions. Supported by the gamma-profile and generalized native classes.
weights	Optional observation-weight expression.
number_of_chosen_alternatives	Expression containing the number of goods chosen in each observation.
consumed_quantities	Named list of observed consumption expressions.
subset	Optional logical expression selecting observations to remove when it is false, using the same semantics as <code>biogeme_model()</code> .
control	Optional <code>biogeme_control()</code> object.

Value

An object of class `biogeme_mdcev_model`.

biogeme_min	<i>Minimum of Biogeme expressions</i>
-------------	---------------------------------------

Description

Minimum of Biogeme expressions

Usage

```
biogeme_min(...)
```

Arguments

... Scalar values or Biogeme expressions.

Value

A Biogeme expression.

biogeme_model	<i>Create a generic Biogeme model</i>
---------------	---------------------------------------

Description

The formula is a neutral R expression. It is compiled once into a native Biogeme expression graph before any estimation or simulation starts.

Usage

```
biogeme_model(
  database,
  formula = NULL,
  weight = NULL,
  probability = NULL,
  simulations = NULL,
  panel_trajectory = FALSE,
  draws = NULL,
  subset = NULL,
  parameter_overrides = NULL,
  control = NULL,
  availability = NULL
)
```

Arguments

database	A biogeme_database object.
formula	A log-likelihood expression, or a named list of formulas. The names log_like and loglike identify the likelihood formula.
weight	Optional observation-weight expression.
probability	Optional probability expression retained for simulation workflows.
simulations	Optional named list of expressions to simulate.
panel_trajectory	If TRUE, aggregate the likelihood using native PanelLikelihoodTrajectory.
draws	Optional draw metadata object or list of draw metadata.
subset	Optional logical expression selecting observations to retain.
parameter_overrides	Optional named list of native parameter replacements, keyed by the original Beta names.
control	Optional <code>biogeme_control()</code> object stored with the model.
availability	Optional named list of availability expressions. This metadata is used by native post-estimation operations such as the null log-likelihood calculation for generic catalog models.

Details

formula is the native log-likelihood expression. A named list may contain log_like (or loglike) and additional expressions, but a generic model must still provide a likelihood, probability, or simulation expressions. simulations is a named list evaluated only when `simulate()` is called.

Value

An object of class `biogeme_model`.

Examples

```
database <- biogeme_database("demo", data.frame(choice = c(1, 2), x = c(1, 2)))
probability <- logit_probability(
  utilities = list(`1` = 0, `2` = biogeme_beta("b") * variable("x")),
  alternative = variable("choice")
)
model <- biogeme_model(
  database,
  formula = logzero(probability),
  simulations = list(probability = probability)
)
model
```

biogeme_model_parameters

Return the parameter definitions in a model

Description

Return the parameter definitions in a model

Usage

```
biogeme_model_parameters(model)
```

Arguments

model A Biogeme model.

Value

A data frame of parameter definitions.

```
biogeme_native_parameter_names
```

Return parameter names collected from the compiled native expression

Description

Unlike `biogeme_model_parameters()`, this operation also sees parameters generated by native helpers such as segmentation and parameters in every catalog branch. When `configuration_id` is supplied, native Biogeme first resolves that configuration and the returned names describe the flattened selected expression.

Usage

```
biogeme_native_parameter_names(  
  model,  
  configuration_id = NULL,  
  model_name = "rbiogeme_parameters",  
  controls = list()  
)
```

Arguments

<code>model</code>	A Biogeme model.
<code>configuration_id</code>	Optional exact native catalog configuration ID.
<code>model_name</code>	Temporary native model name used while compiling.
<code>controls</code>	Named native Biogeme controls.

Value

A list with all, free, and fixed character vectors.

```
biogeme_panel_database
```

Construct a panel database in one call

Description

Construct a panel database in one call

Usage

```
biogeme_panel_database(name, data, panel_id)
```

Arguments

name	Database name.
data	Numeric data frame.
panel_id	Panel identifier column.

Value

A biogeme_database object.

biogeme_prior	<i>Declare a native Bayesian prior</i>
---------------	--

Description

Creates a data-only prior descriptor. The Python bridge compiles it into a native PyMC prior before estimation; R callbacks are not used by the sampler.

Usage

```
biogeme_prior(distribution = c("normal", "student_t"), sigma = 5, nu = 5)
```

Arguments

distribution	Prior distribution: "normal" or "student_t".
sigma	Positive scale parameter.
nu	Positive degrees of freedom for a Student-t prior.

Value

A declarative biogeme_prior object.

biogeme_python	<i>Initialize and return the Python interpreter used by rbiogeme</i>
----------------	--

Description

Initialize and return the Python interpreter used by rbiogeme

Usage

```
biogeme_python()
```

Value

A reticulate Python configuration object. If no interpreter was selected, reticulate may provision the configured native requirement in its managed environment.

biogeme_sampling_partition

Define a native Biogeme sampling partition

Description

Stores the strata and sample sizes used by Biogeme's sampling-of-alternatives API. The segments must cover full_set, and no sample size may exceed the size of its segment.

Usage

```
biogeme_sampling_partition(segments, sample_sizes, full_set = NULL)
```

Arguments

segments	A non-empty list of disjoint integer alternative-ID vectors.
sample_sizes	One positive sample size for each segment.
full_set	Optional integer vector containing the complete partition universe.

Value

A declarative biogeme_sampling_partition object. Native Biogeme performs the actual random sampling when the model is estimated.

See Also

[sampled_alternatives_model\(\)](#)

biogeme_segmentation *Define a discrete parameter segmentation*

Description

Define a discrete parameter segmentation

Usage

```
biogeme_segmentation(variable, mapping, reference = NULL)
```

Arguments

variable	A data-variable name or expression.
mapping	Named mapping from integer values to segment names.
reference	Optional reference segment name.

Value

A biogeme_segmentation specification.

biogeme_segmentation_catalogs

Build synchronized catalogs for possible parameter segmentations

Description

This is the structural counterpart of native Biogeme's `segmentation_catalogs()`. It only enumerates catalog names and builds neutral expression nodes; native Biogeme still creates the segmented parameters and evaluates the selected specification.

Usage

```
biogeme_segmentation_catalogs(
  generic_name,
  beta_parameters,
  potential_segmentations,
  maximum_number,
  selected_name = NULL
)

segmentation_catalogs(
  generic_name,
  beta_parameters,
  potential_segmentations,
  maximum_number,
  selected_name = NULL
)
```

Arguments

`generic_name` Shared controller name.

`beta_parameters` Non-empty list of `biogeme_beta()` expressions.

`potential_segmentations` Non-empty list of segmentation objects.

`maximum_number` Maximum number of segmentations in one option.

`selected_name` Optional segmentation specification selected before native compilation.

Value

A list of synchronized catalog expressions, one per beta.

`biogeme_setup`*Automatically prepare the native Biogeme runtime*

Description

This is the recommended first command for a new user. With no `python` argument, `reticulate` provisions an isolated managed environment containing the configured Biogeme requirement when the runtime is first initialized. If `python` is supplied, that existing interpreter is selected instead and must already contain the requested Biogeme package. In both cases the function returns the same actionable status object as `biogeme_check()`.

Usage

```
biogeme_setup(  
    python = NULL,  
    biogeme_requirement = NULL,  
    debug = NULL,  
    verbose = interactive()  
)
```

Arguments

<code>python</code>	Optional existing Python executable. If omitted, use the <code>reticulate</code> -managed environment when no user-managed environment has been selected.
<code>biogeme_requirement</code>	Python requirement to provision or verify. Defaults to <code>biogeme==3.3.5</code> .
<code>debug</code>	If <code>TRUE</code> , preserve the Python traceback in Biogeme error conditions.
<code>verbose</code>	If <code>TRUE</code> , print the setup status and any corrective actions.

Value

An object of class `biogeme_check`. Its `ready` element is `TRUE` when the runtime can be used for model operations.

See Also

`biogeme_check()`, `biogeme_config()`

boxcox	<i>Box–Cox transformation</i>
--------	-------------------------------

Description

Box–Cox transformation

Usage

```
boxcox(x, lambda)
```

Arguments

x	Expression to transform.
lambda	Box–Cox exponent expression.

Value

A Biogeme expression.

catalog_configuration_ids	<i>Return catalog configuration identifiers using native Biogeme</i>
---------------------------	--

Description

Return catalog configuration identifiers using native Biogeme

Usage

```
catalog_configuration_ids(
  model,
  model_name = "rbiogeme_catalog",
  controls = list(),
  control = NULL
)
```

Arguments

model	A biogeme_model containing catalog expressions.
model_name	Native Biogeme model name.
controls	Named list of native Biogeme controls.
control	Optional biogeme_control() object; an alias for controls.

Value

Character vector of exact native configuration identifiers.

check_derivatives	<i>Check analytical derivatives against native finite differences</i>
-------------------	---

Description

This delegates to the public `BIOGEME.check_derivatives()` operation. The complete expression tree is compiled first; no R callback is evaluated by the derivative checker.

Usage

```
check_derivatives(
  model,
  model_name = "rbiogeme_model",
  controls = list(),
  control = NULL,
  verbose = FALSE
)
```

Arguments

<code>model</code>	A <code>biogeme_model</code> .
<code>model_name</code>	Native Biogeme model name.
<code>controls</code>	Named native Biogeme controls.
<code>control</code>	Optional <code>biogeme_control()</code> object; an alias for controls.
<code>verbose</code>	Whether native Biogeme should print the comparison.

Value

A list containing the native function value, analytical and finite difference gradients and Hessians, and their error vectors.

check_monte_carlo_stability	<i>Run native post-estimation Monte Carlo draw-stability diagnostics</i>
-----------------------------	--

Description

The model is compiled once and the supplied fixed estimate is passed to native `BIOGEME.check_monte_carlo_stability()`. Native Biogeme evaluates the objective and gradient at fresh draw designs and writes its YAML checkpoint and Markdown report.

Usage

```

check_monte_carlo_stability(
  model,
  fit,
  model_name = "rbiogeme_model",
  controls = list(),
  output_directory = NULL,
  basename = NULL,
  resume = TRUE,
  control = NULL
)

```

Arguments

model	A biogeme_model.
fit	A biogeme_fit obtained from the same model.
model_name	Native Biogeme model name.
controls	Named native Biogeme controls.
output_directory	Explicit directory for the native diagnostic files. It has no default: the operation refuses to write into the working directory when this argument is omitted.
basename	Optional native diagnostic filename prefix.
resume	Whether native Biogeme may resume a compatible checkpoint.
control	Optional <code>biogeme_control()</code> object; an alias for controls.

Value

An object of class `biogeme_monte_carlo_diagnostic` containing the native status, conclusion, recommendation, data, and output paths.

count_number_of_specifications

Count catalog specifications using native Biogeme

Description

Count catalog specifications using native Biogeme

Usage

```

count_number_of_specifications(
  model,
  model_name = "rbiogeme_catalog",
  controls = list(),
  control = NULL
)

```

Arguments

model	A biogeme_model containing catalog expressions.
model_name	Native Biogeme model name.
controls	Named list of native Biogeme controls.
control	Optional <code>biogeme_control()</code> object; an alias for controls.

Value

The native number of catalog configurations, or NULL when native Biogeme cannot determine a count.

cross_nested_logit_correlation

Calculate the native cross-nested-logit error-term correlation matrix

Description

Calculate the native cross-nested-logit error-term correlation matrix

Usage

```
cross_nested_logit_correlation(
  model,
  beta_values = NULL,
  alternatives_names = NULL
)
```

Arguments

model	A biogeme_cross_nested_logit_model.
beta_values	Optional named estimated or starting parameter values.
alternatives_names	Optional named character vector for row/column labels.

Value

A numeric correlation matrix.

cross_nested_logit_model

Define a cross-sectional cross-nested-logit model

Description

Define a cross-sectional cross-nested-logit model

Usage

```
cross_nested_logit_model(
  database,
  choice,
  utilities,
  nests,
  availability = NULL,
  alternative_codes = NULL,
  scale_parameter = NULL,
  control = NULL
)
```

Arguments

database	A biogeme_database object.
choice	Name of the integer-valued choice column.
utilities	Named list of utility expressions.
nests	A cross_nested_nests() specification.
availability	Optional named list of availability expressions.
alternative_codes	Optional named integer vector mapping utility names.
scale_parameter	Optional native CNL scale expression.
control	Optional biogeme_control() object.

Value

An object of class biogeme_cross_nested_logit_model.

cross_nested_log_probability

Construct a native cross-nested-logit log-probability expression

Description

Construct a native cross-nested-logit log-probability expression

Usage

```
cross_nested_log_probability(
  utilities,
  availability,
  nests,
  alternative,
  alternative_codes = NULL,
  scale_parameter = NULL
)
```

Arguments

utilities	Named list of utility expressions.
availability	Optional named list of availability expressions.
nests	A cross_nested_nests() specification.
alternative	Integer-valued alternative or a Biogeme choice expression.
alternative_codes	Optional named integer vector mapping utility names.
scale_parameter	Optional native CNL scale expression.

Value

A Biogeme expression compiled to native `biogeme.models.logcnl`.

cross_nested_nest

Define one cross-nested-logit nest

Description

Define one cross-nested-logit nest

Usage

```
cross_nested_nest(nest_parameter, allocation, name = NULL)
```

Arguments

- nest_parameter Native nest parameter expression or numeric value.
- allocation Named mapping from alternative codes to allocation expressions in this nest.
- name Optional native nest name.

Value

A biogeme_cross_nested_nest specification.

cross_nested_nests	<i>Define the complete cross-nested-logit nest structure</i>
--------------------	--

Description

Define the complete cross-nested-logit nest structure

Usage

cross_nested_nests(choice_set, nests, sparse = FALSE)

Arguments

- choice_set All integer alternative codes.
- nests A non-empty list of [cross_nested_nest\(\)](#) specifications.
- sparse If TRUE, allow structurally zero allocation entries to be omitted. Supplied symbolic allocations are always retained.

Value

A biogeme_cross_nested_nests specification.

cross_nested_probability	<i>Construct a native cross-nested-logit probability expression</i>
--------------------------	---

Description

Construct a native cross-nested-logit probability expression

Usage

```
cross_nested_probability(
  utilities,
  availability,
  nests,
  alternative,
  alternative_codes = NULL
)
```

Arguments

<code>utilities</code>	Named list of utility expressions.
<code>availability</code>	Optional named list of availability expressions.
<code>nests</code>	A <code>cross_nested_nests()</code> specification.
<code>alternative</code>	Integer-valued alternative or a Biogeme choice expression.
<code>alternative_codes</code>	Optional named integer vector mapping utility names.

Value

A Biogeme expression compiled to native `biogeme.models.cnl`.

`cross_nested_sparsity_report`

Summarize the structural sparsity of a CNL nest specification

Description

Summarize the structural sparsity of a CNL nest specification

Usage

```
cross_nested_sparsity_report(nests)
```

Arguments

<code>nests</code>	A <code>cross_nested_nests()</code> specification.
--------------------	--

Value

A data frame with one row per nest and membership counts.

cross_variable	<i>Define a sampled-alternative cross-variable</i>
----------------	--

Description

Represents native Biogeme's CrossVariableTuple. The Python bridge expands the expression after native choice-set sampling; R callbacks are not used during estimation.

Usage

```
cross_variable(name, formula)
```

Arguments

name	Name of the generated cross-variable.
formula	A neutral Biogeme expression involving individual and alternative variables.

Value

A declarative biogeme_cross_variable object.

derive	<i>Symbolically differentiate an expression with respect to a named variable</i>
--------	--

Description

Symbolically differentiate an expression with respect to a named variable

Usage

```
derive(expression, name)
```

```
Derive(expression, name)
```

Arguments

expression	Expression to differentiate.
name	Variable or parameter name.

Value

A Biogeme expression.

`distributed_parameter` *Store a simulated individual-level parameter in Bayesian results*

Description

Wraps a child expression in native Biogeme's `DistributedParameter` node. The wrapper preserves the named variable in Bayesian output and is compiled before native estimation; it is not evaluated by R during sampling.

Usage

```
distributed_parameter(name, expression)
```

Arguments

<code>name</code>	Name used for the stored native variable.
<code>expression</code>	Child Biogeme expression, usually a location parameter plus a scale parameter multiplied by <code>draw()</code> .

Details

This helper is a thin interface to native Biogeme and does not implement a sampling or likelihood engine in R.

Value

A symbolic Biogeme expression compiled to native `DistributedParameter`.

Examples

```
beta <- biogeme_beta("b_time")
random_coefficient <- distributed_parameter(
  "b_time_rnd",
  beta + biogeme_beta("b_time_s", start = 1) * draw("b_time_eps", "NORMAL")
)
format(random_coefficient)
```

`draw` *Create a named Biogeme draw node*

Description

Create a named Biogeme draw node

Usage

```
draw(name, draw_type = "NORMAL")
```

Arguments

name	Draw variable name.
draw_type	Biogeme draw type, such as NORMAL or UNIFORM_HALTON2.

Value

A Biogeme expression.

Elem	<i>Select an expression from a numeric mapping</i>
------	--

Description

Select an expression from a numeric mapping

Usage

```
Elem(mapping, index)
```

```
elem(mapping, index)
```

Arguments

mapping	Named list whose names are integer keys.
index	Numeric or Biogeme expression used as the key.

Value

A Biogeme expression.

estimate	<i>Estimate a Biogeme model</i>
----------	---------------------------------

Description

Estimate a Biogeme model

Usage

```
estimate(
  model,
  model_name = "rbiogeme_model",
  controls = list(),
  starting_values = NULL,
  run_bootstrap = FALSE,
  yaml_file_name = NULL,
  control = NULL
)
```

Arguments

model	A biogeme_model, including a model created by <code>logit_model()</code> or another specialized constructor.
model_name	Output/model name used by Biogeme.
controls	Named list of Biogeme controls.
starting_values	Optional named numeric vector of starting values.
run_bootstrap	Whether to run bootstrap re-estimation.
yaml_file_name	Optional path for Biogeme's standard YAML output.
control	Optional <code>biogeme_control()</code> object; an alias for controls.

Details

`estimate()` always requests a fresh native estimation. It does not load a previous YAML or iteration file implicitly. Use `estimate_or_load()` when explicit result loading or recycling is required.

Value

An object of class `biogeme_fit`.

Examples

```
## Not run:
fit <- estimate(
  model,
  model_name = "demo",
  control = biogeme_control(
    output_directory = tempfile("rbiogeme-estimate-"),
    generate_html = FALSE,
    generate_yaml = FALSE,
    save_iterations = FALSE
  )
)
summary(fit)

## End(Not run)
```

estimate_catalog

Estimate every specification in a native Biogeme catalog

Description

Catalog choices, synchronized controllers, estimation, result summaries, Pareto selection, and parameter comparison are all handled by native Biogeme. The returned object only contains ordinary R representations of those native results.

Usage

```
estimate_catalog(
  model,
  model_name = "rbiogeme_catalog",
  controls = list(),
  quick_estimate = FALSE,
  run_bootstrap = FALSE,
  force = TRUE,
  control = NULL
)
```

Arguments

model	A biogeme_model containing one or more catalog expressions.
model_name	Native Biogeme model name prefix.
controls	Named list of native Biogeme controls.
quick_estimate	Whether to use native quick estimation.
run_bootstrap	Whether to run native bootstrap re-estimation.
force	Whether to estimate afresh rather than recycle native files.
control	Optional biogeme_control() object; an alias for controls.

Value

An object of class `biogeme_catalog_fit` containing native model results, summaries, Pareto-optimal specifications, and LaTeX comparison.

estimate_configuration

Estimate one named native catalog configuration

Description

The configuration identifier is resolved by native `BIOGEME.from_configuration()`. R only supplies the compiled expression tree and receives ordinary serialized estimation results; no R expression or callback is evaluated during estimation.

Usage

```
estimate_configuration(
  model,
  configuration_id,
  model_name = "rbiogeme_configuration",
  controls = list(),
  starting_values = NULL,
  run_bootstrap = FALSE,
```

```

    yaml_file_name = NULL,
    control = NULL
  )

```

Arguments

model	A biogeme_model containing catalog expressions.
configuration_id	Exact native configuration identifier, for example "model_catalog:logit;train_tt_catalog:linear"
model_name	Native Biogeme model name.
controls	Named list of native Biogeme controls.
starting_values	Optional named numeric vector of starting values.
run_bootstrap	Whether to run native bootstrap re-estimation.
yaml_file_name	Optional path for native YAML output.
control	Optional <code>biogeme_control()</code> object; an alias for controls.

Value

An object of class `biogeme_fit` containing native results.

estimate_or_load	<i>Estimate or explicitly load a standard Biogeme result</i>
------------------	--

Description

force = FALSE permits loading the named YAML file; force = TRUE always performs fresh estimation. The default estimate() path never recycles an old result file.

Usage

```

estimate_or_load(
  model,
  yaml_file_name,
  force = FALSE,
  model_name = "rbiogeme_model",
  controls = list(),
  starting_values = NULL,
  run_bootstrap = FALSE
)

```

Arguments

model	A biogeme_model.
yaml_file_name	YAML path used for loading or saving.
force	Whether to force fresh estimation.
model_name	Native Biogeme model name.
controls	Named native controls.
starting_values	Optional named starting values.
run_bootstrap	Whether to run bootstrap estimation.

Value

A biogeme_fit object.

estimate_sampled_alternatives

Estimate a native sampled-alternative model

Description

Regenerates the sampled choice sets with recycling disabled, compiles the sampled likelihood once in native Biogeme, and delegates estimation to the native engine. No Python source is generated by the R interface.

Usage

```
estimate_sampled_alternatives(
  model,
  model_name = "rbiogeme_sampled",
  controls = list(),
  starting_values = NULL,
  run_bootstrap = FALSE,
  control = NULL
)
```

Arguments

model	A [sampled_alternatives_model()] object.
model_name	Native Biogeme model name.
controls	Named native Biogeme controls.
starting_values	Optional named finite numeric vector.
run_bootstrap	Whether to run native bootstrap re-estimation.
control	Optional [biogeme_control()] alias for controls.

Value

A biogeme_fit object with native sampling-context and sampled-file metadata attached.

See Also

[sampled_alternatives_model\(\)](#)

evaluate_biogeme_expression

Evaluate a standalone expression with native Biogeme

Description

The expression is compiled once by the Python bridge. The initial result contains the native function value, gradient, Hessian, and BHHH matrix. If points is supplied, the same compiled native callable evaluates each row with the requested free-parameter values. R does not evaluate the expression locally and no Python object is returned.

Usage

```
evaluate_biogeme_expression(
  expression,
  beta = NULL,
  points = NULL,
  numerically_safe = FALSE,
  use_jit = TRUE
)
```

Arguments

expression	A Biogeme expression.
beta	Optional named numeric vector used for the initial evaluation. When omitted, native Beta starting values are used.
points	Optional data frame or numeric matrix. Its columns are free parameter names and its rows are the parameter vectors for repeated native evaluations.
numerically_safe	Whether to request native numerically safe formulas.
use_jit	Whether to use native JAX just-in-time compilation.

Value

An object of class biogeme_expression_evaluation with initial, free_beta_names, and evaluations components.

 evaluate_biogeme_expression_c

Evaluate one expression with native row-wise or aggregated JAX calculation

Description

This operation delegates to Biogeme's public `get_value_c` calculator. The expression is compiled before the native evaluator runs; R receives only a numeric vector or scalar.

Usage

```
evaluate_biogeme_expression_c(
  model,
  expression,
  beta,
  aggregation = FALSE,
  number_of_draws = 1000L,
  numerically_safe = FALSE,
  use_jit = TRUE
)
```

Arguments

<code>model</code>	A <code>biogeme_model</code> supplying the database.
<code>expression</code>	A Biogeme expression to evaluate.
<code>beta</code>	A <code>biogeme_fit</code> or named numeric parameter vector.
<code>aggregation</code>	Whether to return one native aggregated scalar instead of one value per observation.
<code>number_of_draws</code>	Positive number of native Monte Carlo draws.
<code>numerically_safe</code>	Whether to request native numerically safe formulas.
<code>use_jit</code>	Whether to use native JAX just-in-time compilation.

Value

A numeric vector, or one numeric scalar when `aggregation = TRUE`.

integrate_normal	<i>Integrate an expression over a standard normal random variable</i>
------------------	---

Description

Integrate an expression over a standard normal random variable

Usage

```
integrate_normal(expression, name, number_of_quadrature_points = 30L)
```

Arguments

expression	Integrand expression.
name	Random variable name.
number_of_quadrature_points	Number of quadrature points.

Value

A Biogeme expression.

linear_term	<i>Define one coefficient-variable term for a linear utility</i>
-------------	--

Description

Define one coefficient-variable term for a linear utility

Usage

```
linear_term(beta, x)
```

```
LinearTermTuple(beta, x)
```

Arguments

beta	A biogeme_beta expression.
x	A biogeme_variable expression.

Value

A linear-utility term.

linear_utility	<i>Construct a native linear utility expression</i>
----------------	---

Description

Construct a native linear utility expression

Usage

```
linear_utility(terms)
```

```
LinearUtility(terms)
```

Arguments

terms	A non-empty list of <code>linear_term()</code> objects.
-------	---

Value

A Biogeme expression compiled to native `LinearUtility`.

logit_log_probability	<i>Construct a native logit log-probability expression</i>
-----------------------	--

Description

This is the log-probability counterpart of `logit_probability()`. It is compiled directly to Biogeme's public `models.loglogit` constructor, which preserves the native numerically stable likelihood used by examples such as `Swissmetro b20`.

Usage

```
logit_log_probability(
  utilities,
  availability = NULL,
  alternative,
  alternative_codes = NULL
)
```

Arguments

utilities	Named list of utility expressions keyed by alternatives.
availability	Optional named list of availability expressions.
alternative	Integer code or expression for the observed alternative.
alternative_codes	Optional named integer codes for the utilities.

Value

A Biogeme logit log-probability expression.

logit_model

Define a cross-sectional multinomial logit model

Description

Define a cross-sectional multinomial logit model

Usage

```
logit_model(
  database,
  choice,
  utilities,
  availability = NULL,
  alternative_codes = NULL,
  weight = NULL
)
```

Arguments

database	A biogeme_database object.
choice	Name of the integer-valued choice column.
utilities	Named list of utility expressions, one per alternative.
availability	Optional named list of availability expressions.
alternative_codes	Optional named integer vector mapping utility names to values in the choice column. If omitted, utility names must themselves be integer codes.
weight	Optional observation-weight expression.

Details

The utility list is named by alternative. If the names are not integer codes, pass `alternative_codes` as a named integer vector. Availability and weights may be constants or symbolic expressions.

Value

An object of class `biogeme_logit_model`.

Examples

```

database <- biogeme_database(
  "demo",
  data.frame(choice = c(1, 2), x = c(1, 2))
)
b <- biogeme_beta("b")
model <- logit_model(
  database,
  choice = "choice",
  utilities = list(`1` = 0, `2` = b * variable("x"))
)
model

```

logit_probability	<i>Construct a native logit probability expression</i>
-------------------	--

Description

Construct a native logit probability expression

Usage

```

logit_probability(
  utilities,
  availability = NULL,
  alternative,
  alternative_codes = NULL
)

logit(utilities, availability = NULL, alternative, alternative_codes = NULL)

```

Arguments

utilities	Named list of utility expressions, one per alternative.
availability	Optional named list of availability expressions.
alternative	Integer-valued alternative whose probability is returned, or a Biogeme expression selecting the alternative for each observation.
alternative_codes	Optional named integer vector when utility names are not themselves integer alternative codes.

Value

A probability expression compiled to native `biogeme.models.logit`.

logzero	<i>A numerically safe logarithm that returns zero at zero</i>
---------	---

Description

A numerically safe logarithm that returns zero at zero

Usage

```
logzero(x)

safe_log(x)
```

Arguments

x	A scalar or Biogeme expression.
---	---------------------------------

Value

A Biogeme expression.

mdcev_estimate	<i>Estimate an MDCEV model with native Biogeme</i>
----------------	--

Description

The MDCEV likelihood is generated by the native Biogeme MDCEV class after compilation. This convenience wrapper has the same fresh-estimation semantics as [estimate\(\)](#).

Usage

```
mdcev_estimate(
  model,
  model_name = "rbiogeme_mdcev",
  controls = list(),
  starting_values = NULL,
  run_bootstrap = FALSE,
  yaml_file_name = NULL,
  control = NULL
)
```

Arguments

model	A <code>biogeme_mdcev_model()</code> object.
model_name	Native Biogeme model name.
controls	Named native Biogeme controls.
starting_values	Optional named numeric vector of starting values.
run_bootstrap	Whether to run native bootstrap re-estimation.
yaml_file_name	Optional path for native YAML output.
control	Optional <code>biogeme_control()</code> object.

Value

A `biogeme_fit` object.

mdcev_forecast	<i>Forecast MDCEV consumption using native Biogeme algorithms</i>
----------------	---

Description

Forecast MDCEV consumption using native Biogeme algorithms

Usage

```
mdcev_forecast(
  model,
  fit,
  database = NULL,
  total_budget,
  epsilons = NULL,
  number_of_draws = NULL,
  seed = NULL,
  brute_force = FALSE,
  tolerance_dual = 1e-10,
  tolerance_budget = 1e-10
)
```

Arguments

model	A <code>biogeme_mdcev_model()</code> object.
fit	A fresh <code>mdcev_estimate()</code> result.
database	Optional database used only for forecasting. This permits forecasting a native row subset without changing the estimation fit.
total_budget	Positive total budget.

epsilons	Optional list of Gumbel-draw matrices. If omitted, native draws are generated using number_of_draws and seed.
number_of_draws	Number of draws when epsilons is omitted.
seed	Optional native NumPy seed when draws are generated.
brute_force	Whether to use native brute-force optimization.
tolerance_dual	Native dual-variable tolerance.
tolerance_budget	Native budget-constraint tolerance.

Value

A biogeme_mdcev_forecast object containing R data frames.

mdcev_forecast_describe

Return native pandas description tables for an MDCEV forecast

Description

Return native pandas description tables for an MDCEV forecast

Usage

```
mdcev_forecast_describe(forecast)
```

Arguments

forecast A biogeme_mdcev_forecast object.

Value

A list of ordinary R data frames corresponding to native pandas DataFrame.describe() output.

mdcev_generate_epsilon

Generate native MDCEV error-term draws

Description

Generate native MDCEV error-term draws

Usage

```
mdcev_generate_epsilon(
  model,
  number_of_observations,
  number_of_draws,
  seed = NULL
)
```

Arguments

model	A biogeme_mdcev_model() object.
number_of_observations	Number of database observations.
number_of_draws	Number of Gumbel draws per observation.
seed	Optional native NumPy seed.

Value

A list of numeric matrices, one matrix per observation.

mdcev_parameter_table *Return native Biogeme's estimated-parameter table for an MDCEV fit*

Description

Return native Biogeme's estimated-parameter table for an MDCEV fit

Usage

```
mdcev_parameter_table(fit, variance_covariance_type = NULL)
```

Arguments

fit	A biogeme_fit returned by mdcev_estimate() .
variance_covariance_type	Optional native covariance type, such as "BHHH" or "Bootstrap".

Value

A named list of ordinary R data frames returned by native Biogeme.

mdcev_short_summary	<i>Return native Biogeme's compact MDCEV estimation summary</i>
---------------------	---

Description

Return native Biogeme's compact MDCEV estimation summary

Usage

```
mdcev_short_summary(fit)
```

Arguments

fit A biogeme_fit returned by [mdcev_estimate\(\)](#).

Value

The native Biogeme summary string.

mdcev_validate_forecast	<i>Validate the two native MDCEV forecasting algorithms</i>
-------------------------	---

Description

Validate the two native MDCEV forecasting algorithms

Usage

```
mdcev_validate_forecast(
  model,
  fit,
  database = NULL,
  total_budget,
  epsilons = NULL,
  number_of_draws = NULL,
  seed = NULL,
  tolerance_dual = 1e-13,
  tolerance_budget = 1e-13
)
```

Arguments

model	A <code>biogeme_mdcev_model()</code> object.
fit	A fresh <code>mdcev_estimate()</code> result.
database	Optional database used only for forecast validation.
total_budget	Positive total budget.
epsilons	Optional list of Gumbel-draw matrices.
number_of_draws	Number of native draws when epsilons is omitted.
seed	Optional native NumPy seed when draws are generated.
tolerance_dual	Native dual-variable tolerance.
tolerance_budget	Native budget-constraint tolerance.

Value

TRUE when native validation completes without an exception.

monte_carlo	<i>Monte Carlo average of an expression</i>
-------------	---

Description

Monte Carlo average of an expression

Usage

```
monte_carlo(expression)
```

Arguments

expression	Expression to integrate.
------------	--------------------------

Value

A Biogeme expression.

nested_endogenous_sampling_log_probability

Construct a native nested-logit log probability with endogenous-sampling correction

Description

Construct a native nested-logit log probability with endogenous-sampling correction

Usage

```
nested_endogenous_sampling_log_probability(
  utilities,
  availability,
  nests,
  correction,
  alternative,
  alternative_codes = NULL
)
```

Arguments

utilities	Named list of utility expressions.
availability	Optional named list of availability expressions.
nests	A nested_nests() specification.
correction	Named list of alternative-specific log correction terms.
alternative	Integer-valued alternative or a Biogeme choice expression.
alternative_codes	Optional named integer vector mapping utility names.

Value

A Biogeme expression compiled through native `get_mev_for_nested` and `logmev_endogenous_sampling`.

nested_logit_correlation

Calculate the native nested-logit error-term correlation matrix

Description

Calculate the native nested-logit error-term correlation matrix

Usage

```
nested_logit_correlation(
  model,
  beta_values = NULL,
  alternatives_names = NULL,
  mu = 1
)
```

Arguments

model	A biogeme_nested_logit_model.
beta_values	Optional named estimated or starting parameter values.
alternatives_names	Optional named character vector for row/column labels.
mu	Overall scale parameter passed to native Biogeme.

Value

A numeric correlation matrix.

nested_logit_model	<i>Define a cross-sectional nested-logit model</i>
--------------------	--

Description

Define a cross-sectional nested-logit model

Usage

```
nested_logit_model(
  database,
  choice,
  utilities,
  nests,
  availability = NULL,
  alternative_codes = NULL,
  scale_parameter = NULL,
  control = NULL
)
```

Arguments

database	A biogeme_database object.
choice	Name of the integer-valued choice column.
utilities	Named list of utility expressions.

nests	A <code>nested_nests()</code> specification.
availability	Optional named list of availability expressions.
alternative_codes	Optional named integer vector mapping utility names.
scale_parameter	Optional native scale expression for bottom normalization.
control	Optional <code>biogeme_control()</code> object.

Value

An object of class `biogeme_nested_logit_model`.

`nested_log_probability`

Construct a native nested-logit log-probability expression

Description

Construct a native nested-logit log-probability expression

Usage

```
nested_log_probability(
  utilities,
  availability,
  nests,
  alternative,
  alternative_codes = NULL,
  scale_parameter = NULL
)
```

Arguments

utilities	Named list of utility expressions.
availability	Optional named list of availability expressions.
nests	A <code>nested_nests()</code> specification.
alternative	Integer-valued alternative or a Biogeme choice expression.
alternative_codes	Optional named integer vector mapping utility names.
scale_parameter	Optional native scale expression for bottom normalization.

Value

A Biogeme expression compiled to native `biogeme.models.lognested`.

nested_nest	<i>Define one non-trivial nested-logit nest</i>
-------------	---

Description

Define one non-trivial nested-logit nest

Usage

```
nested_nest(nest_parameter, alternatives, name = NULL)
```

Arguments

nest_parameter	Native nest parameter expression or numeric value.
alternatives	Integer alternative codes contained in the nest.
name	Optional native nest name.

Value

A biogeme_nested_nest specification.

nested_nests	<i>Define the complete nested-logit nest structure</i>
--------------	--

Description

Alternatives not listed in a non-trivial nest are treated by native Biogeme as trivial nests containing one alternative.

Usage

```
nested_nests(choice_set, nests)
```

Arguments

choice_set	All integer alternative codes.
nests	A non-empty list of nested_nest() specifications.

Value

A biogeme_nested_nests specification.

nested_probability	<i>Construct a native nested-logit probability expression</i>
--------------------	---

Description

Construct a native nested-logit probability expression

Usage

```
nested_probability(
  utilities,
  availability,
  nests,
  alternative,
  alternative_codes = NULL
)
```

Arguments

utilities	Named list of utility expressions.
availability	Optional named list of availability expressions.
nests	A nested_nests() specification.
alternative	Integer-valued alternative or a Biogeme choice expression.
alternative_codes	Optional named integer vector mapping utility names.

Value

A Biogeme expression compiled to native `biogeme.models.nested`.

normal_cdf	<i>Normal cumulative distribution function</i>
------------	--

Description

Normal cumulative distribution function

Usage

```
normal_cdf(x)
```

Arguments

x	A scalar or Biogeme expression.
---	---------------------------------

Value

A Biogeme expression.

normal_pdf

Normal probability density function

Description

Normal probability density function

Usage

```
normal_pdf(x)
```

Arguments

x A scalar or Biogeme expression.

Value

A Biogeme expression.

ordered_logit_log_probability

Construct an ordered-logit log-likelihood expression

Description

Constructs a neutral expression node. The complete node is compiled once to Biogeme's public ordered-logit expression before estimation or simulation.

Usage

```
ordered_logit_log_probability(eta, cutpoints, alternative,
  categories = NULL, neutral_labels = numeric(),
  enforce_order = TRUE, eps = 1e-12)
```

Arguments

eta	Latent-index expression.
cutpoints	Non-empty list of cutpoint expressions.
alternative	Observed response expression.
categories	Optional numeric response labels.
neutral_labels	Optional numeric labels with unit contribution.
enforce_order	Whether native Biogeme enforces ordered cutpoints.
eps	Positive numerical lower bound used by native Biogeme.

Value

A Biogeme expression compiled to native OrderedLogLogit.

ordered_probit_log_probability

Construct an ordered-probit log-likelihood expression

Description

Constructs a neutral expression node. The complete node is compiled once to Biogeme's public ordered-probit expression before estimation or simulation.

Usage

```
ordered_probit_log_probability(eta, cutpoints, alternative,
  categories = NULL, neutral_labels = numeric(),
  enforce_order = TRUE, eps = 1e-12)
```

Arguments

eta	Latent-index expression.
cutpoints	Non-empty list of cutpoint expressions.
alternative	Observed response expression.
categories	Optional numeric response labels.
neutral_labels	Optional numeric labels with unit contribution.
enforce_order	Whether native Biogeme enforces ordered cutpoints.
eps	Positive numerical lower bound used by native Biogeme.

Value

A Biogeme expression compiled to native OrderedLogProbit.

panel_likelihood_trajectory

Aggregate an observation-level likelihood over a panel trajectory

Description

Aggregate an observation-level likelihood over a panel trajectory

Usage

```
panel_likelihood_trajectory(expression)
```

Arguments

expression Observation-level likelihood or probability expression.

Value

A Biogeme expression.

pareto_post_processing

Re-estimate the Pareto-optimal models saved by native Biogeme

Description

The Pareto file is read by native ParetoPostProcessing. Re-estimation, summary compilation, model descriptions, and optional Pareto plotting all remain native operations; the returned object contains ordinary R values.

Usage

```
pareto_post_processing(
  model,
  pareto_file_name,
  model_name = "rbiogeme_pareto",
  controls = list(),
  recycle = FALSE,
  plot_file_name = NULL,
  objective_x = 0L,
  objective_y = 1L,
  label_x = NULL,
  label_y = NULL,
  control = NULL
)
```

Arguments

model	A biogeme_model containing the catalog expressions used to create the Pareto file.
pareto_file_name	Path to an existing native Pareto file.
model_name	Native Biogeme model-name prefix for re-estimation.
controls	Named list of native Biogeme controls.
recycle	Whether native Biogeme may recycle complete estimation files. The default is FALSE, matching the Python example.
plot_file_name	Optional path where native Pareto plot output is saved.
objective_x	Zero-based native Pareto objective index for the plot.

objective_y	Zero-based native Pareto objective index for the plot.
label_x	Optional native plot x-axis label.
label_y	Optional native plot y-axis label.
control	Optional <code>biogeme_control()</code> object; an alias for controls.

Value

An object of class `biogeme_pareto_fit` containing native results, the compiled summary, descriptions, Pareto statistics, and plot path.

piecewise	<i>Piecewise-linear expression using native Biogeme naming rules</i>
-----------	--

Description

Piecewise-linear expression using native Biogeme naming rules

Usage

```
piecewise(  
  expression,  
  thresholds,  
  betas = NULL,  
  transform = c("formula", "variable")  
)
```

Arguments

expression	Variable expression.
thresholds	Piecewise thresholds; only endpoints may be NULL.
betas	Optional list of parameter expressions.
transform	Whether to return the full formula or the transformed variable form.

Value

A Biogeme expression.

predict.biogeme_fit *Predict native probabilities or simulation expressions*

Description

Predict native probabilities or simulation expressions

Usage

```
## S3 method for class 'biogeme_fit'
predict(object, newdata = NULL, expressions = NULL, control = NULL, ...)
```

Arguments

object	A biogeme_fit or biogeme_bayesian_fit object.
newdata	Optional numeric data frame or biogeme_database used for a scenario prediction. When a data frame is supplied, the model database's derived-variable, filter, and panel metadata are retained.
expressions	Optional named list of complete symbolic expressions. For logit, nested-logit, and cross-nested-logit fits, omission produces one native probability column per alternative. For a generic model, the model's probability or simulation expressions are used.
control	Optional <code>biogeme_control()</code> object for native simulation.
...	Reserved for future prediction options; unsupported arguments are rejected explicitly.

Details

`predict()` is a convenience wrapper around `simulate()`. It does not calculate probabilities in R and does not refit the model. Use `simulate()` directly when several named quantities, posterior draws, or a custom simulation workflow are needed.

Value

A data frame containing values calculated by native Biogeme.

Examples

```
database <- biogeme_database(
  "demo",
  data.frame(choice = c(1, 2), x = c(1, 2))
)
model <- logit_model(
  database,
  choice = "choice",
  utilities = list(`1` = 0, `2` = biogeme_beta("b") * variable("x"))
)
# After fitting: predict(fit) or predict(fit, newdata = data.frame(...))
```

profile_jax

*Profile native JAX formula evaluation***Description**

The complete expression tree is compiled once, then native Biogeme's CompiledFormulaEvaluator is used for each requested evaluation mode. Each mode is evaluated twice, matching the native profiling example's first call and steady-state measurements. Only ordinary environment and timing summaries cross back to R.

Usage

```
profile_jax(
  model,
  beta_values = NULL,
  cases = list(
    list(
      label = "Function only", gradient = FALSE, hessian = FALSE, bhhh = FALSE
    ),
    list(
      label = "Function + gradient", gradient = TRUE, hessian = FALSE, bhhh = FALSE
    ),
    list(
      label = "Function + gradient + Hessian",
      gradient = TRUE, hessian = TRUE, bhhh = FALSE
    ),
    list(
      label = "Function + gradient + BHHH",
      gradient = TRUE, hessian = FALSE, bhhh = TRUE
    )
  ),
  model_name = "rbiogeme_model",
  controls = list(),
  numerically_safe = FALSE,
  control = NULL
)
```

Arguments

model	A biogeme_model.
beta_values	Named finite numeric vector of parameter values. If NULL, model starting values are used.
cases	A non-empty list of named case lists. Each case has a non-empty label and optional logical gradient, hessian, and bhhh flags.
model_name	Native Biogeme model name.
controls	Named native Biogeme controls.

numerically_safe	Whether native formula evaluation uses numerical-safety transformations.
control	Optional <code>biogeme_control()</code> object; an alias for controls.

Details

Timing values depend on the active Python, JAX, hardware, and thread configuration. The operation is a profiling diagnostic, not an estimation operation.

Value

An object of class `biogeme_jax_profile` containing the native JAX environment and serialized profile data for each case.

See Also

`biogeme_control`

quick_estimate	<i>Estimate a Biogeme model using the native quick-estimation operation</i>
----------------	---

Description

`quick_estimate()` delegates to native `BIOGEME.quick_estimate()`. It returns parameter values and the statistics that native Biogeme makes available without the ordinary post-estimation derivative calculations. Use [save_results\(\)](#) when a standard YAML result is required.

Usage

```
quick_estimate(
    model,
    model_name = "rbiogeme_model",
    controls = list(),
    control = NULL
)
```

Arguments

model	A <code>biogeme_model</code> .
model_name	Output/model name used by Biogeme.
controls	Named list of Biogeme controls.
control	Optional biogeme_control() object; an alias for controls.

Value

An object of class `biogeme_fit`.

random_variable	Create a random variable for native numerical integration
-----------------	---

Description

Create a random variable for native numerical integration

Usage

```
random_variable(name)
```

Arguments

name	Random variable name.
------	-----------------------

Value

A Biogeme expression.

read_results	Read standard Biogeme YAML results
--------------	------------------------------------

Description

Read standard Biogeme YAML results

Usage

```
read_results(filename)
```

Arguments

filename	YAML file generated by Biogeme.
----------	---------------------------------

Value

A biogeme_fit object.

safe_exp	<i>Numerically safe exponential</i>
----------	-------------------------------------

Description

Numerically safe exponential

Usage

```
safe_exp(x)
```

Arguments

x A scalar or Biogeme expression.

Value

A Biogeme expression.

sampld_alternatives_model	<i>Define a sampled-alternative Biogeme model</i>
---------------------------	---

Description

Defines the protocol for a native sampling-of-alternatives model. Sampling, cross-variable expansion, likelihood construction, differentiation, and optimization are performed by native Biogeme through the Python bridge. Sampling is regenerated with recycling disabled by default.

Usage

```
sampld_alternatives_model(
  alternatives,
  individuals,
  choice_column,
  id_column,
  utility,
  partition,
  biogeme_file_name,
  model_type = c("logit", "nested", "cnl"),
  combined_variables = list(),
  mev_partition = NULL,
  mev_sample_sizes = NULL,
  nests = NULL,
  control = NULL
)
```

Arguments

<code>alternatives</code>	Numeric data frame with one row per alternative.
<code>individuals</code>	Numeric data frame with one row per decision maker.
<code>choice_column</code>	Choice column in <code>individuals</code> ; values are alternative IDs.
<code>id_column</code>	Unique alternative-ID column in <code>alternatives</code> .
<code>utility</code>	Complete neutral Biogeme utility expression.
<code>partition</code>	Main [<code>biogeme_sampling_partition()</code>].
<code>biogeme_file_name</code>	Path for the native sampled-data file.
<code>model_type</code>	One of "logit", "nested", or "cnl".
<code>combined_variables</code>	Optional list of [<code>cross_variable()</code>] objects.
<code>mev_partition</code>	Optional partition used for MEV terms.
<code>mev_sample_sizes</code>	Optional MEV sample sizes.
<code>nests</code>	Nested or cross-nested nest object for the selected model type.
<code>control</code>	Optional [<code>biogeme_control()</code>] object.

Value

A `biogeme_sampled_alternatives_model` object containing ordinary R data and a neutral expression specification.

See Also

[estimate_sampled_alternatives\(\)](#)

sampling_segment_sizes

Generate balanced sampling segment sizes

Description

Creates sampling protocol metadata equivalent to native `generate_segment_size()`; native Biogeme still performs validation and sampling.

Usage

```
sampling_segment_sizes(sample_size, number_of_segments)
```

Arguments

<code>sample_size</code>	Total number of alternatives to sample.
<code>number_of_segments</code>	Number of sampling segments.

Value

An integer vector whose entries differ by at most one. Any remainder is assigned to the first segments, matching native Biogeme.

save_results	<i>Save standard Biogeme YAML results</i>
--------------	---

Description

Save standard Biogeme YAML results

Usage

```
save_results(fit, filename)
```

Arguments

fit	A biogeme_fit object.
filename	Destination YAML file.

Value

fit, invisibly.

segment_beta	<i>Create a segmented parameter expression</i>
--------------	--

Description

The generated native parameters follow Biogeme’s public naming contract: <base>_ref for the reference and <base>_diff_<segment> for differences.

Usage

```
segment_beta(beta, segmentations, prefix = "segmented")  
  
segmented_beta(beta, segmentations, prefix = "segmented")
```

Arguments

beta	A biogeme_beta expression.
segmentations	A non-empty list of biogeme_segmentation objects.
prefix	Native segmentation expression prefix.

Value

A Biogeme expression compiled through native Segmentation.

simulate

Simulate named native Biogeme expressions at fixed estimates

Description

Simulate named native Biogeme expressions at fixed estimates

Usage

```
simulate(model, expressions = NULL, beta, control = NULL, database = NULL)
```

Arguments

model	A biogeme_model.
expressions	Optional named list of expressions. When omitted, the model's simulations list is used.
beta	A biogeme_fit or named numeric parameter vector.
control	Optional <code>biogeme_control()</code> object.
database	Optional scenario database. It may be a biogeme_database or a numeric data frame. A data frame inherits the model database's derived-variable, filter, and panel metadata.

Details

expressions is a named list of complete symbolic expressions. When it is omitted, the model's simulations list is used. Native Biogeme evaluates these expressions at the supplied parameter values and returns ordinary R data frames.

Value

A biogeme_simulation object containing a data frame of values.

Examples

```
## Not run:
simulation_model <- biogeme_model(
  database,
  formula = choice_log_probability,
  simulations = list(probability = choice_probability)
)
simulated <- simulate(
  simulation_model,
  beta = fit,
  control = biogeme_control(output_directory = tempfile("rbiogeme-sim-"))
)
as.data.frame(simulated)

## End(Not run)
```

simulate_bayesian	<i>Simulate formulas over Bayesian posterior draws</i>
-------------------	--

Description

Evaluates named simulation formulas over native Bayesian posterior draws and returns the native mean and quantile summaries without exposing PyMC objects.

Usage

```
simulate_bayesian(model, bayesian_results, expressions = NULL,
  percentage_of_draws_to_use = 10, lower_quantile = 0.025,
  upper_quantile = 0.975, control = NULL)
```

Arguments

model	A biogeme_model.
bayesian_results	A biogeme_bayesian_fit or a path to a native NetCDF Bayesian result file.
expressions	Optional named list of simulation expressions.
percentage_of_draws_to_use	Percentage of posterior draws evaluated by native Biogeme.
lower_quantile	Lower posterior-simulation quantile.
upper_quantile	Upper posterior-simulation quantile.
control	Optional biogeme_control() object.

Value

A biogeme_simulation object containing native summaries.

simulate_single_formula	<i>Evaluate one native Biogeme formula as an aggregated scalar</i>
-------------------------	--

Description

This operation delegates to Biogeme's public calculate_single_formula_from_expression evaluator. Unlike [simulate\(\)](#), it returns one native aggregated value, which is useful for panel likelihood formulas.

Usage

```
simulate_single_formula(
    model,
    expression,
    beta,
    number_of_draws,
    seed = NULL,
    numerically_safe = FALSE,
    use_jit = TRUE
)
```

Arguments

model	A biogeme_model.
expression	A Biogeme expression to evaluate.
beta	A biogeme_fit or named numeric parameter vector.
number_of_draws	Positive number of native Monte Carlo draws.
seed	Optional temporary native draw seed.
numerically_safe	Whether to request native numerically safe formulas.
use_jit	Whether to use native JAX just-in-time compilation.

Value

One numeric scalar returned by native Biogeme.

sqrt.biogeme_expression

Square root of a Biogeme expression

Description

Square root of a Biogeme expression

Usage

```
## S3 method for class 'biogeme_expression'
sqrt(x)
```

Arguments

x	A scalar or Biogeme expression.
---	---------------------------------

Value

A Biogeme expression.

swissmetro_b01b_model *Build the segmented linear-utility Swissmetro specification from b01b*

Description

This uses native LinearUtility and Segmentation during bridge compilation. The generated parameter names are asc*_ref and asc*_diff_*, matching the Python example.

Usage

```
swissmetro_b01b_model(
    database,
    user_notes =
        paste0("Example of a logit model with three alternatives: Train, Car and ",
              "Swissmetro. Same as 01logit and introducing LinearUtility and ",
              "automatic segmentation of parameters.")
)
```

Arguments

database A database prepared by [swissmetro_data\(\)](#).
user_notes Optional notes stored in the native estimation result.

Value

A biogeme_logit_model.

swissmetro_data *Prepare the Swissmetro database used by Biogeme's examples*

Description

The transformation is deliberately expressed as native database operations. It removes rows with PURPOSE other than 1 or 3 and rows with CHOICE == 0, then defines the cost, availability, and /100 scaled variables used by plot_b01a_logit.py. Row order and original row IDs are retained. Set panel = TRUE to declare ID as the panel identifier.

Usage

```
swissmetro_data(
    data,
    name = "swissmetro",
    panel = FALSE,
    filter_purpose = TRUE
)
```

```
prepare_swissmetro(
  data,
  name = "swissmetro",
  panel = FALSE,
  filter_purpose = TRUE
)
```

Arguments

data	Numeric Swissmetro data frame.
name	Database name.
panel	Whether to declare the ID panel identifier.
filter_purpose	Whether to retain only observations with PURPOSE equal to 1 or 3, in addition to removing CHOICE == 0. Set this to FALSE for native examples whose read_data() keeps all purposes.

Value

A biogeme_database specification.

swissmetro_mnl_model *Build the baseline Swissmetro MNL specification*

Description

Build the baseline Swissmetro MNL specification

Usage

```
swissmetro_mnl_model(database)
```

Arguments

database	A database prepared by swissmetro_data() .
----------	--

Value

A biogeme_logit_model.

validate	<i>Validate a model using native Biogeme cross-validation</i>
----------	---

Description

Validate a model using native Biogeme cross-validation

Usage

```
validate(model, fit, folds = 5L, groups = NULL, seed = NULL, control = NULL)
```

Arguments

model	A biogeme_model.
fit	A biogeme_fit from the same model.
folds	Number of validation folds.
groups	Optional grouping column.
seed	Optional seed controlling native random fold assignment.
control	Optional <code>biogeme_control()</code> object.

Value

A native-bridge validation result.

validate_model	<i>Validate a model before estimation</i>
----------------	---

Description

Model construction, database operations, and native Biogeme model creation are checked without running an optimizer or writing estimation results. The returned diagnostics are ordinary R values; the native model object is created and discarded inside the bridge.

Usage

```
validate_model(model, control = NULL)
```

Arguments

model	A biogeme_model.
control	Optional <code>biogeme_control()</code> object. It is used only when native model construction needs a control value.

Details

This is specification validation, not cross-validation. Use `validate()` after estimation when the goal is out-of-sample fold evaluation.

Value

An object of class `biogeme_model_validation` containing the native validation status, database information, formula names, and parameter count.

Examples

```
## Not run:
database <- biogeme_database(
  "demo",
  data.frame(choice = c(1, 2), x = c(1, 2))
)
model <- logit_model(
  database,
  choice = "choice",
  utilities = list(`1` = 0, `2` = biogeme_beta("b") * variable("x"))
)
validate_model(model)

## End(Not run)
```

variable	<i>Create a Biogeme data variable</i>
----------	---------------------------------------

Description

Create a Biogeme data variable

Usage

```
variable(name)
```

Arguments

name	Column name in the Biogeme database.
------	--------------------------------------

Details

`variable()` creates a symbolic reference. It does not extract an R column or calculate a vector immediately. Use it in arithmetic and model expressions after the corresponding database column has been defined.

Value

A variable expression.

Examples

```
x <- variable("income")  
b <- biogeme_beta("b_income")  
b * x + 1
```

Index

- * **models**
 - ordered_logit_log_probability, [67](#)
 - ordered_probit_log_probability, [68](#)
- abs.biogeme_expression, [5](#)
- assisted_specification, [6](#)
- bayesian_estimate, [7](#)
- bayesian_posterior_mean_by_observation, [8](#)
- bayesian_stored_variables, [8](#)
- biogeme_beta, [9](#)
- biogeme_catalog, [10](#)
- biogeme_catalog_controller, [10](#)
- biogeme_catalog_controller(), [10](#)
- biogeme_check, [11](#)
- biogeme_check(), [22](#), [34](#)
- biogeme_confidence_intervals, [11](#)
- biogeme_config, [12](#)
- biogeme_config(), [22](#), [34](#)
- biogeme_control, [13](#)
- biogeme_control(), [7](#), [12](#), [27](#), [28](#), [35–39](#), [46–48](#), [57](#), [64](#), [70](#), [71](#), [73](#), [78](#), [83](#)
- biogeme_database, [15](#)
- biogeme_database(), [27](#)
- biogeme_database_columns, [15](#)
- biogeme_database_define_variable, [16](#)
- biogeme_database_extract_rows, [17](#)
- biogeme_database_filtered_row_count, [17](#)
- biogeme_database_has_column, [18](#)
- biogeme_database_is_panel, [18](#)
- biogeme_database_materialize, [19](#)
- biogeme_database_nrow, [19](#)
- biogeme_database_panel, [20](#)
- biogeme_database_remove, [20](#)
- biogeme_database_row_ids, [21](#)
- biogeme_database_segmentation, [22](#)
- biogeme_diagnostics, [22](#)
- biogeme_draws, [23](#)
- biogeme_error_details, [23](#)
- biogeme_general_statistics, [24](#)
- biogeme_generic_alt_specific_catalogs, [24](#)
- biogeme_max, [25](#)
- biogeme_mdcev_model, [26](#)
- biogeme_mdcev_model(), [57](#), [59](#), [61](#)
- biogeme_min, [27](#)
- biogeme_model, [28](#)
- biogeme_model(), [27](#)
- biogeme_model_parameters, [29](#)
- biogeme_model_parameters(), [30](#)
- biogeme_native_parameter_names, [30](#)
- biogeme_panel_database, [30](#)
- biogeme_prior, [31](#)
- biogeme_prior(), [9](#)
- biogeme_python, [31](#)
- biogeme_sampling_partition, [32](#)
- biogeme_segmentation, [32](#)
- biogeme_segmentation_catalogs, [33](#)
- biogeme_setup, [34](#)
- biogeme_setup(), [11](#)
- boxcox, [35](#)
- catalog(biogeme_catalog), [10](#)
- catalog_configuration_ids, [35](#)
- catalog_controller
 - (biogeme_catalog_controller), [10](#)
- check_derivatives, [36](#)
- check_monte_carlo_stability, [36](#)
- count_number_of_specifications, [37](#)
- cross_nested_log_probability, [40](#)
- cross_nested_logit_correlation, [38](#)
- cross_nested_logit_model, [39](#)
- cross_nested_nest, [40](#)
- cross_nested_nest(), [41](#)
- cross_nested_nests, [41](#)
- cross_nested_nests(), [39](#), [40](#), [42](#)
- cross_nested_probability, [41](#)

- cross_nested_sparsity_report, 42
- cross_variable, 43
- database_define_variable
 - (biogeme_database_define_variable), 16
- database_extract_rows
 - (biogeme_database_extract_rows), 17
- database_generate_segmentation
 - (biogeme_database_segmentation), 22
- database_remove
 - (biogeme_database_remove), 20
- define_variable
 - (biogeme_database_define_variable), 16
- Derive (derive), 43
- derive, 43
- distributed_parameter, 44
- draw, 44
- Elem, 45
- elem (Elem), 45
- estimate, 45
- estimate(), 56
- estimate_catalog, 46
- estimate_configuration, 47
- estimate_or_load, 48
- estimate_or_load(), 46
- estimate_sampled_alternatives, 49
- estimate_sampled_alternatives(), 76
- evaluate_biogeme_expression, 50
- evaluate_biogeme_expression_c, 51
- generic_alt_specific_catalogs
 - (biogeme_generic_alt_specific_catalogs), 24
- integrate_normal, 52
- linear_term, 52
- linear_utility, 53
- LinearTermTuple (linear_term), 52
- LinearUtility (linear_utility), 53
- logit (logit_probability), 55
- logit_log_probability, 53
- logit_model, 54
- logit_model(), 46
- logit_probability, 55
- logit_probability(), 53
- logzero, 56
- mdcev_estimate, 56
- mdcev_estimate(), 57, 59–61
- mdcev_forecast, 57
- mdcev_forecast_describe, 58
- mdcev_generate_epsilon, 59
- mdcev_model (biogeme_mdcev_model), 26
- mdcev_parameter_table, 59
- mdcev_short_summary, 60
- mdcev_validate_forecast, 60
- monte_carlo, 61
- nested_endogenous_sampling_log_probability, 62
- nested_log_probability, 64
- nested_logit_correlation, 62
- nested_logit_model, 63
- nested_nest, 65
- nested_nest(), 65
- nested_nests, 65
- nested_nests(), 62, 64, 66
- nested_probability, 66
- normal_cdf, 66
- normal_pdf, 67
- ordered_logit_log_probability, 67
- ordered_probit_log_probability, 68
- panel_likelihood_trajectory, 68
- pareto_post_processing, 69
- piecewise, 70
- predict.biogeme_fit, 71
- prepare_swissmetro (swissmetro_data), 81
- profile_jax, 72
- quick_estimate, 73
- random_variable, 74
- rbiogeme-package, 4
- read_results, 74
- remove_observations
 - (biogeme_database_remove), 20
- reticulate::py_require(), 12
- run_assisted_specification
 - (assisted_specification), 6
- safe_exp, 75

- safe_log(logzero), [56](#)
- sampld_alternatives_model, [75](#)
- sampld_alternatives_model(), [32](#), [50](#)
- sampling_segment_sizes, [76](#)
- save_results, [77](#)
- save_results(), [73](#)
- segment_beta, [77](#)
- segmentation_catalogs
 - (biogeme_segmentation_catalogs),
[33](#)
- segmented_beta(segment_beta), [77](#)
- simulate, [78](#)
- simulate(), [29](#), [71](#), [79](#)
- simulate_bayesian, [79](#)
- simulate_single_formula, [79](#)
- sqrt.biogeme_expression, [80](#)
- swissmetro_b01b_model, [81](#)
- swissmetro_data, [81](#)
- swissmetro_data(), [81](#), [82](#)
- swissmetro_mnl_model, [82](#)

- validate, [83](#)
- validate(), [84](#)
- validate_model, [83](#)
- variable, [84](#)